
Unifor

T2

Grupo D: Problema C (Network)

Alunos: Gabriel Rocha (2410451), Nicolas Marcelino (2420508)
Orientador: Prof. Me. Ricardo Carubbi
Disciplina: Resolução de Problemas com Grafos
Centro de Ciências Tecnológicas - CCT
Universidade de Fortaleza - UNIFOR

Introdução

- Dado um grafo não direcionado e conectado, em que cada vértice representa uma **cidade** e cada aresta representa uma **linha telefônica**, é necessário descobrir quantas cidades são críticas.
- Uma cidade é crítica se, ao removê-la e eliminar todas as suas conexões, a rede fica **desconectada**, impedindo que algumas das cidades restantes se comuniquem entre si.

Objetivo: **contar** quantas cidades são críticas.

Baseado na descrição do problema *Network* na plataforma Online Judge.
Disponível em: https://onlinejudge.org/external/3/315.pdf.

Modelagem
Algoritmo de componentes conexas

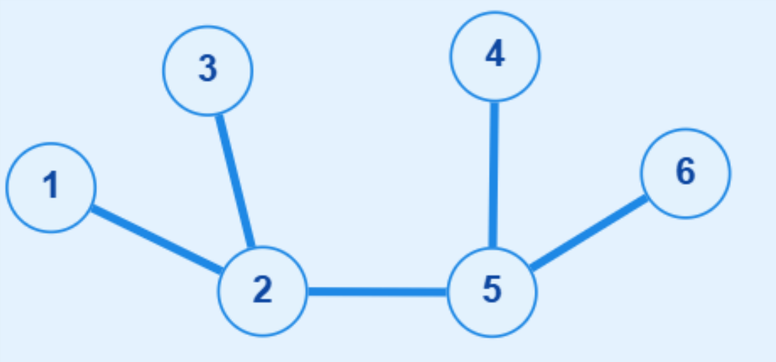
- Objetivo: mapear as componentes conexas e seus vértices.
- Estruturas principais:

boolean[] marked (visitados)

int[] id (relaciona vértice e sua componente)

int count (contagem de componentes)

- *Algoritmo aplicado: DFS*



Grafo baseado no caso de teste disponibilizado pela plataforma. Fonte: elaborado pelos autores com auxílio do Graph Online.

Baseado na descrição do problema *Network* na plataforma Online Judge.
Disponível em: https://onlinejudge.org/external/3/315.pdf.

Solução

A remoção de um vértice r é lógica: G não é alterado, e a classe CC apenas ignora r durante as visitas, simulando o grafo G - r

```
def __init__(self, G, r):
    self.marked = [False for _ in range(G.V)]
    self.id = [0 for _ in range(G.V)]
    self._size = [0 for _ in range(G.V)]
    self.count = 0

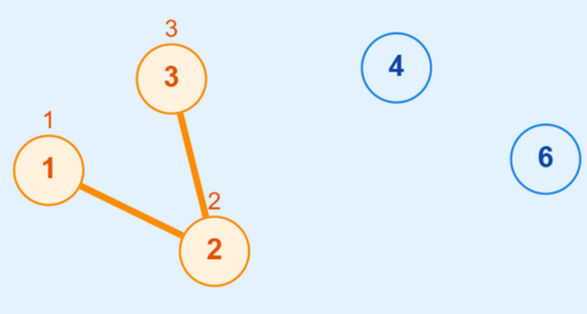
    for s in range(G.V):
        if not self.marked[s] and s != r:
            self.dfs(G, s, r)
            self.count += 1
```

Impede que r seja usado como ponto de partida de uma DFS.
Sem essa condição, r iniciaria uma componente própria e seria contado em count.

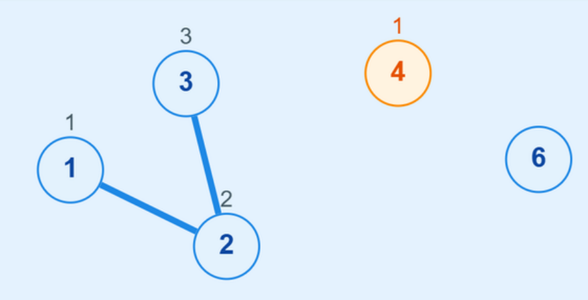
```
def dfs(self, G, v, r):
    self.marked[v] = True
    self.id[v] = self.count
    self._size[self.count] += 1
    for w in G.adj[v]:
        if not self.marked[w] and w != r:
            self.dfs(G, w, r)
```

Impede que r seja alcançado como vizinho de outro vértice. Sem essa condição, a DFS entraria em r por qualquer aresta que chegasse nele e continuaria a partir dele, atravessando para outras partes do grafo. Isso anularia a remoção.

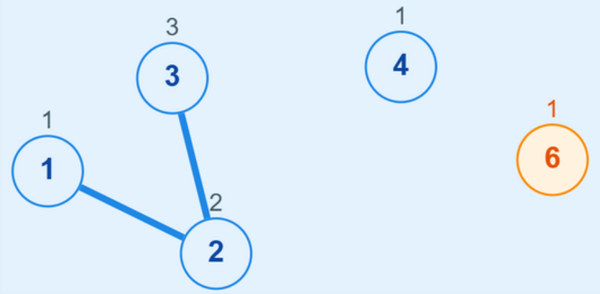
Simulação resumida do algoritmo
Remoção do vértice 5



dfs(G, 1, 5)
marked = [True, True, True, False, False, False]
id = [1, 1, 1, 0, 0, 0]
count = 1



dfs(G, 4, 5)
marked = [True, True, True, True, False, False]
id = [1, 1, 1, 2, 0, 0]
count = 2



dfs(G, 6, 5)
marked = [True, True, True, True, False, True]
id = [1, 1, 1, 2, 0, 3]
count = 3

Complexidade

A solução é ótima: roda em tempo linear, O(V + E), o mesmo custo de apenas ler a entrada, enquanto o critério direto custa O(V · (V + E)).

Aspecto	Complexidade	Descrição
Tempo (critério eficiente)	**O(V + E)**	Cada vértice é visitado uma vez pela DFS e cada lista de adjacência é percorrida uma vez (soma dos graus = 2E). Contagem final dos críticos: O(V). Na instância principal: V + 2E = 6 + 10 = 16 operações.
Tempo (critério direto)	**O(V · (V + E))**	Remove cada vértice e roda uma DFS. Na instância principal: 6 DFS, cada uma sobre no máximo 5 vértices.
Leitura da entrada	**O(V + E) = O(E)**	Como a rede é conexa, E ≥ V − 1, então O(V + E) = O(E).

V: Número de locais (vértices).
E: Número de linhas da rede (arestas).

Base teórica: material disponibilizado pelo Prof. Me. Ricardo Carubbi, baseado em Sedgewick e Wayne (2014). Disponível em: https://github.com/carubbi/RPG.